ON THE NINU SUPERMARKET

Retail POS Web Application

Project Report

FastAPI + SQLite + SQLAlchemy + Jinja + Bootstrap + Chart.js

Done by:

- **Abdel-Rahman Gamal Hussein**
- **Mahmoud Ahmed Arafa**
- **Abdullah Abdel-Raheem**
- **Ali Khaled**
- **Moaz Reda**

➔ **Under Supervision of Dr. Samar Awad**

➔ **Faculty of Artificial Intelligence Engineering**

Contents

1. Project Overview

2. Project Objectives

3. Technology Stack

4. Database Design Summary

5. Entity Relationship Diagram

6. Application Structure

7. Main System Modules

8. Authentication and Role Access

9. Dashboard and Reports

10. User Interface Improvements

11. Testing and Validation

12. How to Run the Project

13. Limitations and Future Improvements

14. Conclusion

1. Project Overview

This project is a complete local web application for a supermarket point-of-sale system. It was built for On The Ninu Supermarket to manage daily retail operations such as products, suppliers, sales, payments, customers, returns, employees, branches, and purchase orders.

The main idea of the project is to connect the database design with a real working web interface. Instead of only keeping the ERD and SQL scripts, the system provides screens where a user can log in, manage data, create sales, view reports, and track the business from a dashboard.

The application runs locally and does not depend on cloud services or paid tools. This makes it suitable for testing, university submission, and future extension into a larger retail system.

Important project note

The latest web and theme improvements were added without changing the original business ERD. The business schema stayed the same. The only extra application table is user_account, which is required for login and session-based access.

2. Project Objectives

The project was developed to achieve the following objectives:

- Convert the corrected retail ERD into a working database structure.
- Build SQLAlchemy models that represent all project entities and relationships.
- Provide CRUD screens for the main database tables.
- Add a login system with role-based access for different employee types.
- Create a clear dashboard that summarizes the current state of the supermarket.
- Add useful reports such as sales, inventory, purchase orders, and returns.
- Keep the project easy to run on a local computer using SQLite.
- Keep a SQL Server version of the schema for database documentation and future deployment.

3. Technology Stack

Layer	Tool	Purpose
Backend framework	FastAPI	Used to build the web application and REST API endpoints.
Database	SQLite	Used as the default local database for development and testing.
ORM	SQLAlchemy	Maps Python classes to database tables and manages relationships.
Templates	Jinja2	Renders dynamic HTML pages from FastAPI.
Frontend	HTML, CSS, JavaScript, Bootstrap	Builds the responsive user interface and forms.
Charts	Chart.js	Displays dashboard charts and report graphs.
Security	Session authentication + password hashing	Protects pages and separates access by role.

4. Database Design Summary

The database follows a retail/POS structure. It is divided into four practical areas: administration, catalog and supply, sales and customers, and authentication.

Area	Main tables
Admin / operations	role, role_permission, branch, department, employee, register, shift
Catalog / supply	supplier, category, product, inventory_movement, discount, discount_product, purchase_order, po_item
Sales / customer	customer, loyalty_account, points_transaction, sale, sale_item, payment, sale_return, sale_return_item
Authentication	user_account

4.1 Main Relationships

- A branch contains registers and departments.
- An employee belongs to a department and has a role.
- A supplier provides products and can have purchase orders.
- A category contains products, and categories can also have parent categories.
- A sale is made by an employee through a register and may be linked to a customer.
- A sale contains many sale items and can have one or more payments.
- A customer can have a loyalty account and points transactions.
- A sale return belongs to a sale and contains returned products.
- A purchase order belongs to a supplier and contains purchase order items.

5. Entity Relationship Diagram

The following diagram shows the organized database layout for the project. The diagram is grouped to make the connections easier to follow: admin and operations, catalog and supply, sales and customer, and authentication.

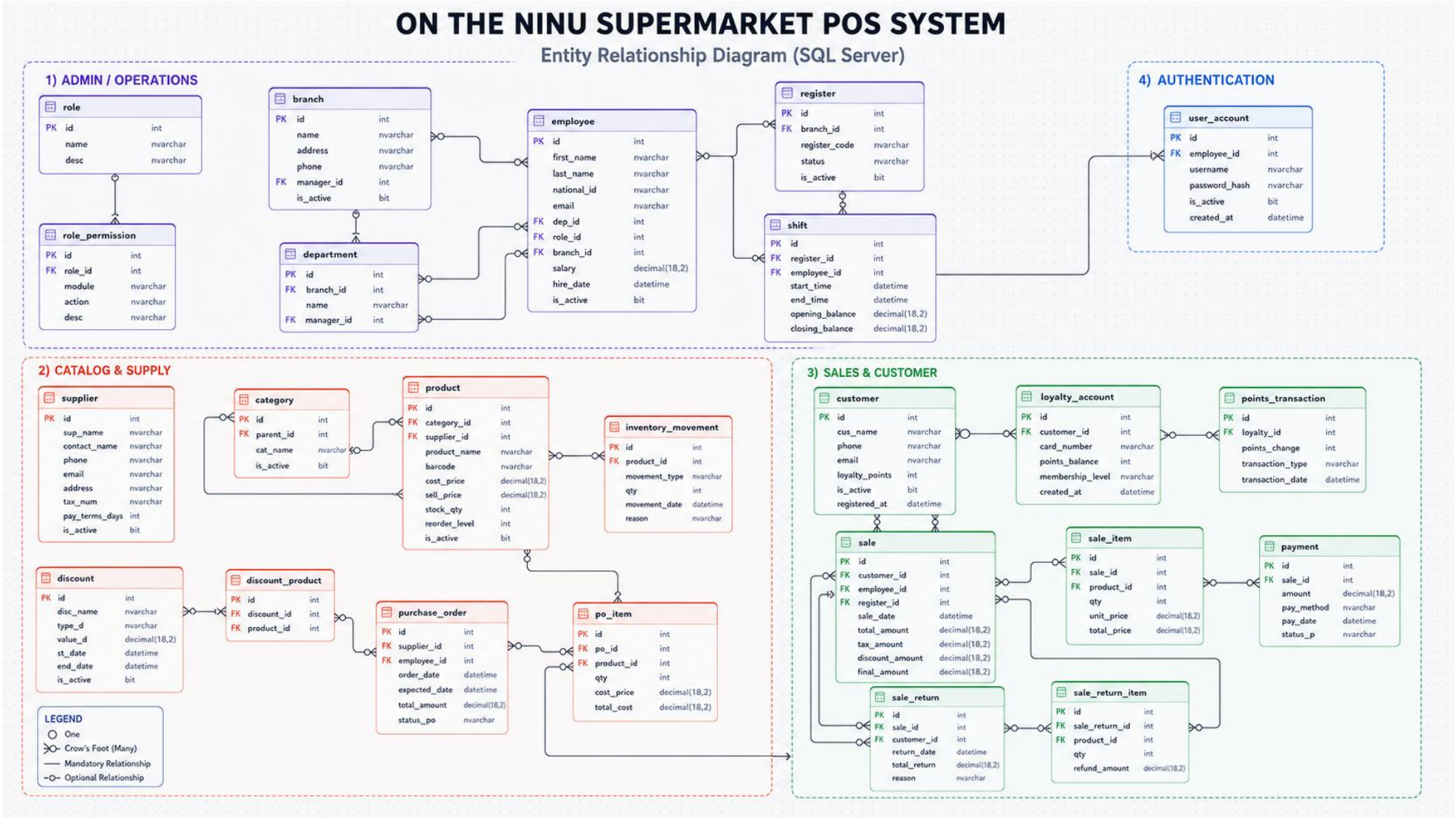

Figure 1. Organized SQL Server style ERD for the On The Ninu POS system.

6. Application Structure

The project files are organized in a simple structure so the backend, database scripts, templates, and static files are separated clearly.

```
retail_pos_app/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── crud.py
│   ├── auth.py
│   ├── routes/
│   ├── templates/
│   └── static/
├── scripts/
│   ├── init_db.py
│   ├── seed_data.py
│   ├── retail_pos_erd_aligned_sqlite_schema_seed.sql
│   └── retail_pos_erd_aligned_sqlserver_schema_seed.sql
├── requirements.txt
├── README.md
├── .env.example
└── retail_pos.db
```

File	Use in the project
app/main.py	Starts the FastAPI application and connects routes, templates, sessions, and static files.
app/models.py	Contains SQLAlchemy models for the corrected ERD tables.
app/routes/web.py	Handles dashboard pages, generic CRUD screens, and web navigation.
app/routes/api.py	Contains REST API routes for database entities.
app/routes/reports.py	Builds report pages and CSV export logic.
app/auth.py	Contains login, logout, password verification, and role access helpers.
scripts/init_db.py	Creates or recreates the local SQLite database.
scripts/seed_data.py	Loads sample data and creates login users.

7. Main System Modules

Product and category management: The system stores products, categories, barcodes, cost price, selling price, stock quantity, reorder level, and supplier links.

Supplier and purchase orders: Suppliers are connected to purchase orders and purchase order items. This helps track incoming stock.

Sales module: Sales are recorded with a cashier/employee, register, customer, tax, discount, final amount, items, and payment details.

Returns module: Returned products are stored through sale_return and sale_return_item, with return reason and refund amount.

Inventory tracking: Inventory movements record stock changes from purchases, sales, returns, and adjustments.

Customer loyalty: Customers can have loyalty accounts, card numbers, membership levels, and points transactions.

Admin management: Branches, departments, registers, employees, roles, and permissions are handled through the admin/operations part of the database.

8. Authentication and Role Access

The application includes a login page and uses session-based authentication. After logging in, each user sees the system according to their role. This makes the project more realistic because supermarket staff should not all have the same permissions.

Role	Access
Admin	Full access to all pages and records.
Manager	Dashboard, products, sales, customers, payments, and reports.
Cashier	Sales, payments, customers, and returns.
Inventory Clerk	Products, suppliers, inventory movements, and purchase orders.

8.1 Demo Users

Username	Password	Role
admin	Admin@12345	Admin
manager	Manager@12345	Manager
cashier	Cashier@12345	Cashier
inventory	Inventory@12345	Inventory Clerk

The passwords are for local testing only. In a real deployment, the demo users should be removed or changed before using the system with real data.

9. Dashboard and Reports

The dashboard gives a quick view of the business. It is not just a welcome page; it reads data from the database and summarizes important numbers and recent records.

- Total products
- Total customers
- Total sales
- Total employees
- Total suppliers
- Pending purchase orders
- Low stock products
- Recent sales

9.1 Reports Included

Report	What it is used for
Sales report	Shows sales totals, dates, payment information, and supports date filtering and CSV export.
Inventory report	Shows current stock, low-stock products, suppliers, and reorder levels.
Purchase orders report	Shows supplier orders, expected dates, status, and total amounts.
Returns report	Shows sale returns, reasons, refund amounts, and returned products.

9.2 Charts

Chart.js is used for visual summaries. The dashboard can display sales over time and payment method distribution. This helps a manager understand the system quickly without opening every table manually.

10. User Interface Improvements

The web interface was improved to look closer to a real supermarket system. The goal was to make the project easier to use and more presentable, without changing the database schema or ERD.

- A branded On The Ninu Supermarket theme using supermarket-style green and orange colors.
- A grouped sidebar so pages are not shown as one long unorganized list.
- A working mobile menu button with sidebar open and close behavior.
- Dashboard cards and cleaner spacing for important statistics.
- CRUD tables with search, filters, actions, and confirmation before delete.
- Relationship dropdowns for foreign key fields instead of forcing the user to type IDs manually.
- Better login page and clear logout button.

10.1 POS Sales Screen

A POS-style sales screen was added to make the system feel more like a real cashier workflow. Instead of creating sale and sale item records separately, the cashier can build a cart, choose products, calculate totals, select payment method, and then save the sale.

After saving a sale, the system can show a receipt/invoice page that can be printed. This is one of the most important improvements because it connects the database design with the real use case of a supermarket.

11. Testing and Validation

The project was checked mainly through local testing with the SQLite database. The database was initialized, seed data was inserted, and the application was started with Uvicorn.

Test area	Result
Database creation	init_db.py --drop recreates the database tables.
Seed data	seed_data.py inserts sample branches, employees, products, suppliers, customers, sales, and login users.
Login	Demo users can log in and reach their allowed pages.
CRUD pages	Tables can be listed, searched, added, edited, and deleted when safe.
Foreign keys	The app blocks unsafe deletes when a record is referenced by another table.
Reports	Reports read database data and export CSV where available.

12. How to Run the Project

The project is made to run locally. The normal setup commands are shown below.

```
unzip retail_pos_app.zip
cd retail_pos_app

python -m venv .venv
source .venv/bin/activate
# Windows PowerShell: ./.venv/Scripts/Activate.ps1

pip install -r requirements.txt
cp .env.example .env

python scripts/init_db.py --drop
python scripts/seed_data.py

python -m uvicorn app.main:app --reload
```

After running the application, open the following URL in the browser:

```
http://127.0.0.1:8000
```

The API documentation can also be opened from:

```
http://127.0.0.1:8000/docs
```

12.1 Windows PowerShell Note

If PowerShell blocks the virtual environment activation script, run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass in the same PowerShell window, then activate the environment using ./.venv/Scripts/Activate.ps1.

13. Limitations and Future Improvements

The current version is suitable for local use and project demonstration. However, it can still be improved if the system is later used in a real business environment.

- Add barcode scanner support on the POS screen.
- Add PDF invoice export in addition to print view.
- Add stronger audit logs for every create, update, and delete action.
- Add backup and restore options for the SQLite database.
- Add branch-level reporting and cashier shift closing reports.
- Add product images and receipt logo upload.
- Move from SQLite to SQL Server or PostgreSQL if the system becomes multi-user in production.
- Add more detailed permission control using role_permission records instead of only role names.

14. Conclusion

The On The Ninu Supermarket POS project successfully turns the corrected retail ERD into a working local web application. The system includes database tables, relationships, CRUD screens, login, role access, dashboard cards, charts, reports, and seed data.

The strongest part of the project is that it does not stop at database design. It connects the schema to real supermarket screens such as sales, products, customers, payments, returns, and inventory. The added UI theme and organized sidebar also make the project easier to present and use.

Overall, the project is a solid base for a retail POS system. It is ready for local demonstration, and it can be extended later into a more advanced production system.

Appendix A. Database Tables Summary

Table	Purpose
supplier	Stores supplier details such as name, contact, phone, email, address, tax number, and payment terms.
role	Stores employee role names such as Admin, Manager, Cashier, and Inventory Clerk.
role_permission	Stores modules and actions allowed for each role.
branch	Stores supermarket branch information and branch manager link.
register	Stores POS register machines linked to branches.
department	Stores departments inside branches.
employee	Stores staff information, department, role, salary, and activity status.
shift	Stores register shifts with opening and closing balances.
category	Stores product categories and parent categories.
product	Stores product data, category, supplier, prices, stock, and reorder level.
inventory_movement	Tracks product stock movements from sales, purchases, returns, and adjustments.
discount	Stores discount campaigns with type, value, start date, and end date.
discount_product	Links discounts to products.
customer	Stores customer details and loyalty point summary.
loyalty_account	Stores loyalty card number, balance, and membership level.
points_transaction	Stores each change in loyalty points.
sale	Stores sale header data such as customer, employee, register, totals, tax, and discount.

sale_item	Stores products sold inside each sale.
payment	Stores payment amount, method, date, and status for a sale.
sale_return	Stores sale return header and return reason.
sale_return_item	Stores returned products and refund amounts.
purchase_order	Stores purchase order header data linked to supplier and employee.
po_item	Stores purchase order lines and costs.
user_account	Stores login username, password hash, and employee link.

Appendix B. Database Versions

The project keeps two schema versions:

- SQLite schema: used by the FastAPI application during local development.
- SQL Server schema: kept as a separate DBMS version for SSMS and future migration.

The SQL Server version uses SQL Server style data types such as INT, NVARCHAR, DECIMAL(18,2), BIT, and DATETIME2(0). The SQLite version is kept separately because SQLite has different type handling and is easier to run locally.